

Privacy-First Email Prioritization

with Active Learning on Small Language Models

Local Qwen2.5-1.5B + post-hoc calibration + interactive AL — no email content leaves the device.

Hansheng Zeng · Jiaming Zhao · JJ Sandhu

4-minute summary · 100-email Enron pilot

Triage 200 emails before coffee — without sending them to a cloud

- Information workers receive ~120 emails/day.
- Rule-based filters are brittle; cloud LLM APIs require uploading content.
- Privacy regulations (HIPAA, attorney-client, internal HR) forbid that route.

OUR ANGLE

Run a 1.5B local SLM, accept that it's miscalibrated, and close the gap with two cheap, on-device moves:

Post-hoc calibration

(no fine-tuning, no SLM re-query)

Interactive active learning

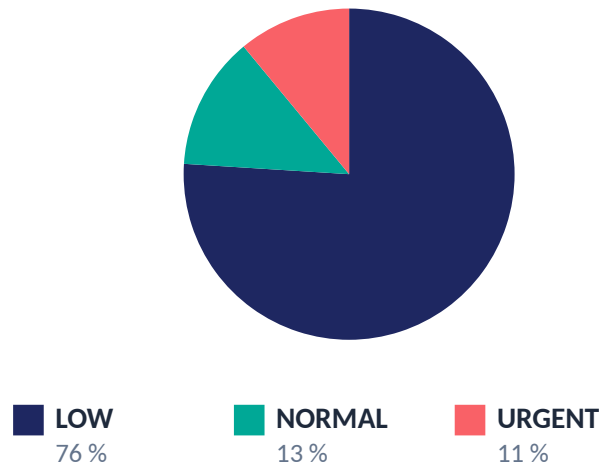
(prompt updates from real corrections)

Privacy-first means no email content ever leaves the device.

Dataset: 100 emails, severely skewed

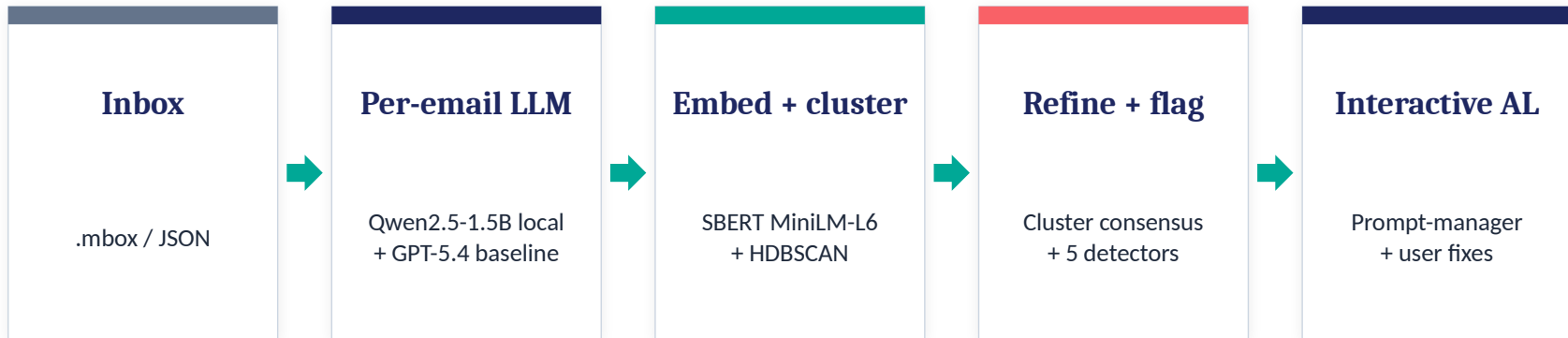
Source	Enron corpus, arnold-j inbox
Pilot size	100 emails, manually labelled
Labels	{urgent, normal, low}
Annotators	team consensus (3 reviewers)
Splits	5-fold stratified CV for calibration

Ground-truth class distribution



Key consequence: with 76% LOW, a constant-LOW classifier already scores $\text{Acc}=0.76$. We must report macro-F1 and Spearman ρ alongside accuracy.

End-to-end pipeline — runs entirely on a laptop



FEEDBACK LOOP

Corrections rewrite the prior + rotate demos in the next SLM call (history-compressed prompt-manager).

End-to-end latency: ~58 s for 100 emails on CPU (Qwen2.5-1.5B + SBERT + HDBSCAN). Zero email content sent to any cloud service.

Three required functionalities — three modules

01

Per-email LLM scoring

Qwen2.5-1.5B (local) emits {urgency, score 0–100, deadline, signals}. GPT-5.4 used as a cloud upper bound only.

`slm_scorer_v2.py`

02

Cluster-based refinement

Sentence-BERT (MiniLM-L6) embeds all emails; HDBSCAN finds dense clusters; predictions snap toward cluster consensus.

`clustering_refinement.py`

03

Missing-context detection

5 regex detectors (attachment, forwarded thread, prior conversation, ambiguous deadline, external link) surface where the SLM is acting on partial info.

`missing_context_detector.py`

Pipeline runs end-to-end on a laptop. Total latency: ~58 s for 100 emails on CPU.

Post-hoc calibration: macro-F1 0.28 → 0.39

Qwen2.5-1.5B over-predicts urgent (40 / 100 vs. 11 truth).
We fit a 13-feature calibrator on 5-fold CV — no fine-tuning,
no extra SLM calls.

+39%macro-F1
(rel. lift)**0.43**accuracy
(LR balanced)**0**extra LLM
calls needed

Method	Acc	mF1	ρ
SLM raw (Qwen2.5)	0.28	0.28	0.24
+ LogReg (balanced)	0.43	0.39	0.26
+ LogReg (unbalanced)	0.76	0.41	0.23
+ Random forest	0.72	0.36	0.25
+ Gradient boosting	0.70	0.41	0.17
GPT-5.4 (cloud, upper bound)	0.71	0.58	0.62

*Class-balanced LogReg lifts macro-F1 +39% while preserving rank ρ .
Unbalanced models exploit the 76% LOW prior — accuracy lies on skewed data.*

Stage 2: cluster the inbox before scoring

How it works

100 emails → 7 clusters + noise

1

Embed every email

Sentence-BERT all-MiniLM-L6 → 384-dim vector (≤ 25 ms / email on CPU).

2

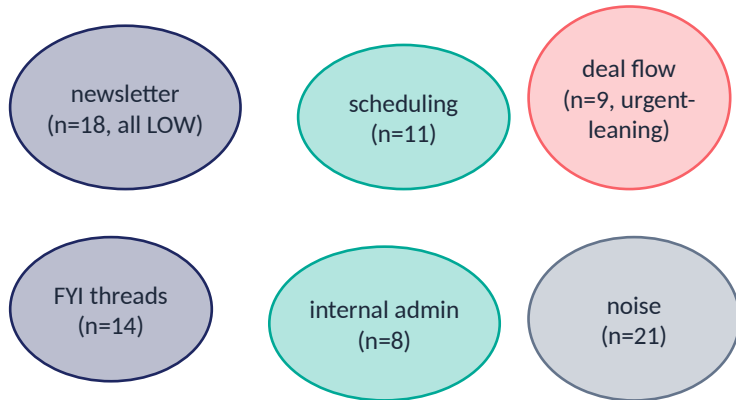
Density-based clustering

HDBSCAN with min_cluster_size = 4. No k to tune, noise points handled natively.

3

Refine outlier predictions

If $\geq 75\%$ of a cluster shares a label, snap minority predictions toward the consensus.



Result: newsletter cluster snaps 4 over-predicted urgent emails back to LOW; refinement is most useful as a sanity check on the SLM.

Surprise: random sampling matches entropy AL

Real CLI loop: user rates u/n/l + free-text note. Corrections feed a prompt-manager that:

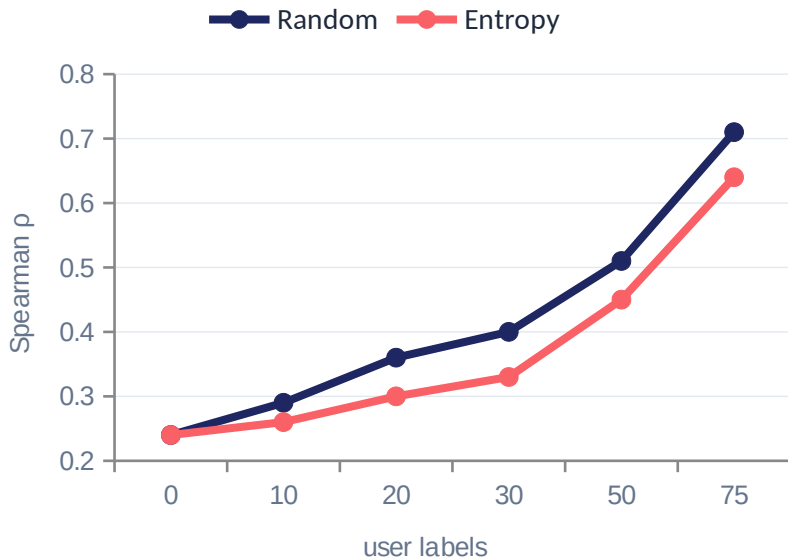
- rotates class-balanced demos (cap 8)
- rewrites the prior-summary sentence
- re-scores only when prior shifts > 5%

THE FINDING

Random $p@50 = 0.51$ vs. Entropy $p@50 = 0.45$

Cause: 3-bin SLM confidence collapses 100 emails into 9 buckets — entropy AL has no dynamic range to exploit.

Feedback efficiency (n=5 seeds)



Implication: cluster-stratified random is a defensible default until SLMs expose real per-class log-probs.

Missing-context: 5 detectors, honest evaluation

Five regex detectors

Missing attachment	<i>see attached / PFA / enclosed</i>	0.85
Forwarded thread	<i>--- Forwarded Message ---</i>	0.75
Prior conversation	<i>as we discussed / per our call</i>	0.70
Ambiguous deadline	<i>ASAP / EOW / earliest convenience</i>	0.65
External link	<i>https:// (whitelist: Zoom, Teams)</i>	0.50

HONEST EVALUATION

No human annotation of "missing-context" — we use a proxy oracle (GT=urgent $\wedge$ SLM=low).

On 100 emails the proxy yields **1 positive**

→ heuristic flags 42 / 100

→ SLM built-in flag flags 3 / 100

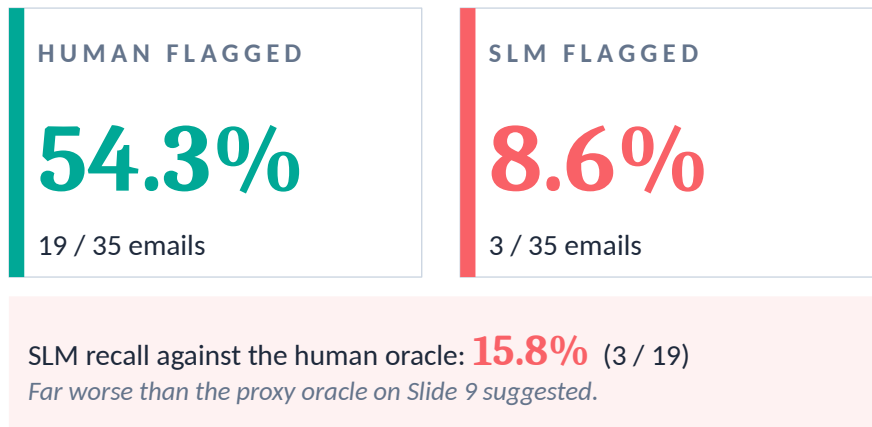
Reported as a lower-bound, not a final precision number. Future work: ≥ 200 binary annotations.

UI surfaces concrete evidence spans ("as we discussed last week") — not just a yes/no flag.

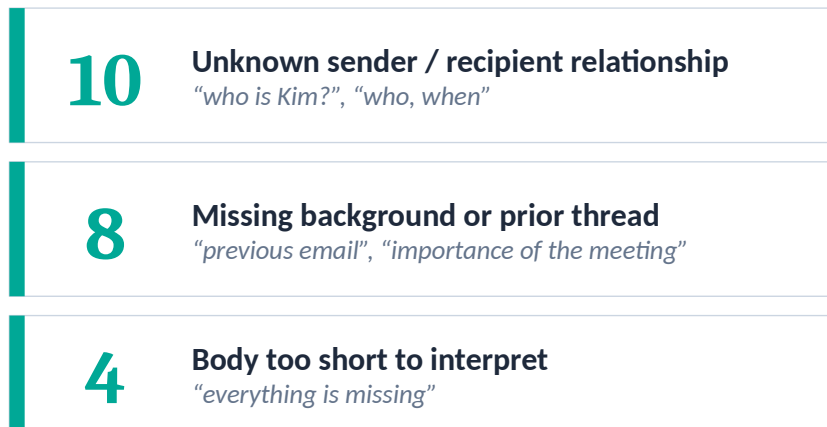
The SLM misses the contexts that matter

Interactive AL loop, entropy sampling, 7 iterations × batch 5 = **35 emails** annotated with both an urgency label and a binary “missing context?” judgment + free-text reason.

The detection gap

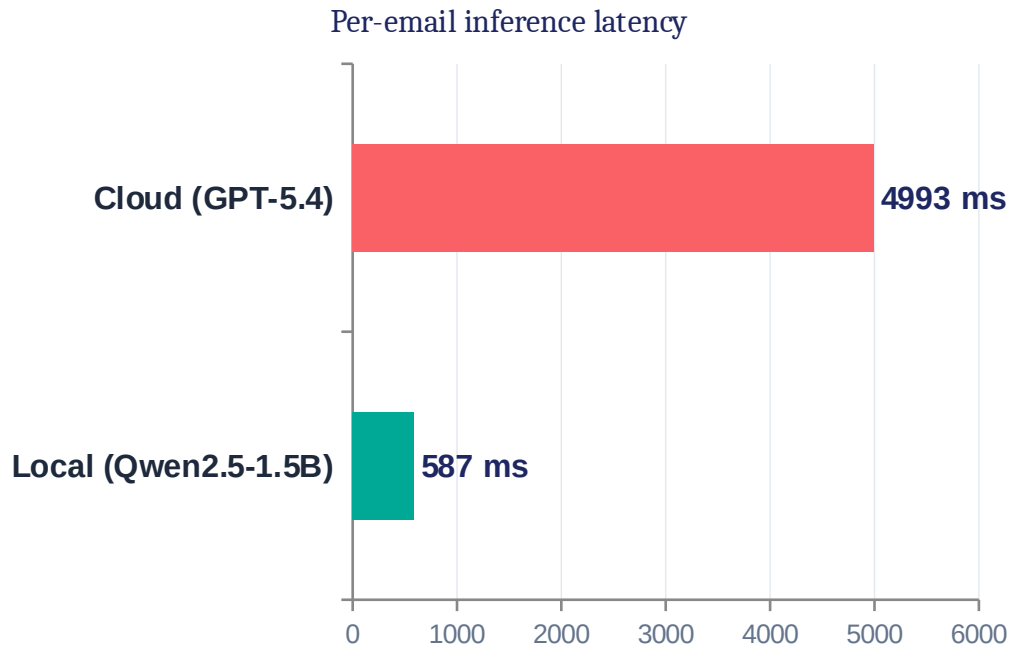


What context is actually missing



Takeaway: the SLM equates “short or unusual format” with “missing context” — it over-flags machine emails and misses the relational gaps that humans actually feel.

Counterintuitive: local SLM is 8.5× faster than cloud



8.5×

faster on a CPU laptop than
calling GPT-5.4 over HTTPS

0

bytes of email content sent off-
device — privacy-first by
construction

\$0

marginal cost per inbox triage
(no API tokens)

*The cloud's edge in this setting is **quality**, not **speed** — which reframes the cost-benefit calculation when privacy is a constraint.*

Demo & takeaways

demo.html — 117 KB, no server

Single file. Opens with file:// — zero network, zero analytics.

- Sort by predicted urgency, date, or uncertainty
- Switch view: SLM raw / GPT-5.4 / GT / your fixes
- One-click correction with optional note
- AL toggle surfaces top-10 most-uncertain
- Live readout of running per-user prior

KEY TAKEAWAYS

+39%

macro-F1 from on-device calibration alone (no FT)

0.51

best p at 50 labels (random > entropy AL)

8.5×

local SLM faster per email than GPT-5.4 (587 ms vs 4 993 ms)

0

bytes of email content sent to the cloud

Where the numbers are still soft

1. Pilot is 100 emails

Rare-class recall standard errors ≥ 0.15 .
Scale to 500–1 000 emails across
multiple Enron users (parse_data.py
reads \$ENRON_MAILDIR).

2. SLM has no logprobs

Switch to llama.cpp logprobs to make
uncertainty AL pay off. Until then, cluster-
stratified random is the production
default.